Performance Overhead of FIPS 203/204/205 Post-Quantum Cryptography in BFT Consensus

Abstract

This paper reports measured performance overhead of NIST post-quantum cryptographic standards (ML-DSA-65, ML-KEM-768, SLH-DSA-128s) integrated into a HotStuff-based BFT consensus protocol. We instrument a 4-validator network running the complete block production pipeline — key generation, transaction signing, consensus voting, signature verification, and parallel transaction execution — and compare classical (Ed25519) against post-quantum (ML-DSA-65) configurations. End-to-end throughput with ML-DSA-65 reaches 10,260 TPS with 9.75 ms block latency on commodity hardware. The primary bottleneck is transaction signing (83.2% of total time); verification overhead is negligible (1.3x). We validate implementation correctness against 117 official NIST ACVP test vectors with byte-exact output matching. Results are fully reproducible from the open-source implementation.

1. Introduction

NIST finalized three post-quantum cryptographic standards in August 2024: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA). NIST IR 8547 establishes deprecation of classical algorithms by 2030 and prohibition by 2035. While resource estimates for quantum attacks on elliptic curve cryptography continue to decrease — Gidney and Ekera reduced RSA-2048 factoring to 20 million noisy qubits in 8 hours [1], and Google Quantum AI's 2026 estimates place secp256k1 ECDLP at under 500,000 physical qubits in 9 minutes [2] — no published work reports measured performance of these standards in a production blockchain consensus system.

Prior work on PQC performance focuses on isolated cryptographic operations [3][4] or theoretical protocol analysis [5]. Blockchain-specific studies address quantum threats [6] but not deployment performance. This gap matters because BFT consensus protocols amplify cryptographic costs: each consensus round requires O(n) signatures and O(n^2) verifications across validators.

This paper addresses the gap with measurements from a deployed system using real cryptographic operations throughout the consensus and transaction processing pipeline.

2. System Architecture

The system under test implements a DAG-based HotStuff BFT consensus protocol with Delegated Proof-of-Stake validator selection. The cryptographic module is isolated in a separate crate with FIPS 140-3 compliant state machine (4 states, 3 transitions, power-up self-tests).

2.1 Cryptographic Primitives

Primitive	Standard	Purpose	Public Key	Signature
ML-DSA-65	FIPS 204	Block signatures, BFT votes, certificates	1,952 B	3,309 B
SLH-DSA-SHAKE-128s	FIPS 205	Backup signatures (hash-based)	32 B	7,856 B
ML-KEM-768	FIPS 203	TLS key exchange (hybrid with X25519)	1,184 B	ct: 1,088 B
Ed25519	RFC 8032	Legacy/ hybrid classical component	32 B	64 B
SHA3-256	FIPS 202	Block hashing, content hashing	—	32 B

2.2 Consensus Protocol

HotStuff BFT with 3 phases (Prepare, PreCommit, Commit). Quorum threshold: 2f+1 out of 3f+1 validators. Each phase requires f+1 valid signatures. A single consensus round produces 9 signatures (3 phases x 3 validators) and 9 verifications.

2.3 Hybrid Mode

Following ANSSI recommendation [7], the system supports dual signatures (classical + post-quantum) on signed envelopes. Both signatures must verify for the envelope to be accepted. The hybrid TLS layer uses X25519 + ML-KEM-768 via rustls-post-quantum.

3. Methodology

3.1 Test Environment

- Hardware: Apple M-series, 8 cores, 16 GB RAM
- Toolchain: Rust nightly, release profile (optimizations enabled)
- Crypto library: PQClean (C reference implementations via FFI)
- Measurement: `std::time::Instant` (monotonic clock)
- Iterations: 20 blocks x 100 transactions per benchmark run
- Validators: 4 (minimum BFT with f=1)

3.2 Pipeline Instrumentation

The benchmark measures three pipeline phases independently:

1. **BFT Consensus**: Key generation, vote signing (9 signs per round), vote verification (9 verifies per round), state machine transitions.
2. **Transaction Signing**: Each transaction signed by the submitter using the configured algorithm.
3. **Execution**: Parallel transaction execution against in-memory world state with MVCC conflict detection.

Wall-clock time recorded per phase per block. Total time includes all phases plus overhead (serialization, memory allocation).

3.3 ACVP Validation

Implementation correctness verified against official NIST ACVP-Server test vectors (github.com/usnistgov/ACVP-Server). Results:

Algorithm	Test	Vectors	Result
ML-DSA-65	keyGen	25	25/25 byte-exact
ML-DSA-65	sigGen (deterministic)	30	30/30 byte-exact
ML-DSA-65	sigVer (pure mode)	12	12/12 correct
ML-KEM-768	keyGen	25	25/25 byte-exact
ML-KEM-768	encapsulation	25	25/25 byte-exact

4. Results

4.1 Isolated Cryptographic Operations

100 iterations per algorithm (SLH-DSA: 10), release build.

Operation	Ed25519	ML-DSA-65	SLH-DSA-128s	ML-DSA/ Ed25519
KeyGen (us)	9.2	39.1	456,048	4.2x
Sign (us)	9.1	90.5	352,797	9.9x

Operation	Ed25519	ML-DSA-65	SLH-DSA-128s	ML-DSA/ Ed25519
Verify (us)	19.6	25.5	352.5	1.3x
Public key (B)	32	1,952	32	61.0x
Signature (B)	64	3,309	7,856	51.7x

SLH-DSA-128s signing is 3,880x slower than ML-DSA-65, confirming its role as emergency backup only.

4.2 End-to-End Pipeline

20 blocks, 100 transactions per block, 4 validators, real cryptography.

Metric	Ed25519	ML-DSA-65	Overhead
Throughput (TPS)	64,561	10,260	6.3x
Block rate (blocks/s)	646	103	6.3x
Block latency (ms)	1.55	9.75	6.3x

4.3 Time Breakdown

Phase	Ed25519 (ms)	Ed25519 (%)	ML-DSA-65 (ms)	ML-DSA-65 (%)
BFT consensus	6.20	20.0	28.67	14.7
TX signing	20.73	66.9	162.22	83.2
Execution	3.62	11.7	3.53	1.8

Transaction signing dominates in both configurations. The shift from 66.9% to 83.2% reflects the 9.9x signing overhead of ML-DSA-65 while execution cost remains constant.

4.4 Scaling Behavior

ML-DSA-65, 10 blocks per configuration.

Txs/Block	TPS	Block Latency (ms)	Bottleneck
10	4,442	2.25	BFT
50	8,956	5.58	Signing
100	10,041	9.96	Signing
200	10,790	18.54	Signing
500	11,014	45.40	Signing

TPS plateaus around 11,000 as signing becomes the dominant cost. At low transaction counts (<50), BFT consensus overhead dominates. The crossover occurs near 30 transactions per block.

4.5 Hybrid Signature Overhead

ANSSI-compliant dual signature (Ed25519 primary + ML-DSA-65 secondary), 50 iterations.

Operation	Classical (us)	Hybrid (us)	Overhead
Sign	12.6	85.4	6.8x
Verify	18.9	64.2	3.4x
Bandwidth per envelope	96 B	5,325 B	55.5x

4.6 Key Exchange (TLS)

ML-KEM-768, 100 iterations.

Operation	Latency (us)
KeyGen	9.8
Encapsulate	9.7
Decapsulate	8.9
Handshake overhead	+2,208 B

4.7 Block Size Impact

Estimated block size with N endorsements (signature + payload hash + public key per endorsement).

Endorsements	Ed25519 (KB)	ML-DSA-65 (KB)	Ratio
1	0.4	8.5	23.8x
5	1.0	29.2	29.7x
10	1.8	55.1	31.2x
20	3.3	106.8	32.1x

5. Discussion

5.1 Verification is Not the Bottleneck

The 1.3x verification overhead of ML-DSA-65 contradicts the assumption that PQC significantly impacts consensus validation. In HotStuff BFT, validators verify O(n) signatures per phase — at 25.5 us per verify, a 100-validator network adds only 2.55 ms per phase, well within typical network round-trip times.

5.2 Signing Dominates

Transaction signing consumes 83.2% of pipeline time with ML-DSA-65. This cost is per-transaction and scales linearly. Parallelizing transaction signing across available cores is the most direct optimization path — the current implementation signs sequentially.

5.3 Bandwidth Considerations

ML-DSA-65 signatures (3,309 B) are 51.7x larger than Ed25519 (64 B). For a block with 100 transactions and 5 endorsements each, the bandwidth difference is approximately 1.5 MB (ML-DSA) vs 30 KB (Ed25519). At 103 blocks/second, this produces approximately 155 MB/s of signature data. This is within the capacity of modern datacenter networks but may constrain geographic distribution over slower links.

5.4 Comparison with Existing Systems

No directly comparable measurements exist in the literature. QRL uses XMSS (stateful hash-based signatures) but has not published BFT consensus benchmarks. Algorand deploys FALCON for State Proofs but not for transaction signatures. Neither Bitcoin nor Ethereum has deployed any PQC algorithm.

6. Reproducibility

All benchmarks are executed via:

```
cargo test --release --test e2e_throughput -- --nocapture
cargo test --release --test pqc_benchmark -- --nocapture
```

Source code, test vectors, and benchmark harness are available in the repository. ACVP validation vectors sourced from github.com/usnistgov/ACVP-Server.

References

[1] C. Gidney and M. Ekera, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” Quantum, vol. 5, p. 433, 2021.

[2] Google Quantum AI, “Resource estimates for cryptographically relevant quantum computers,” 2026.

[3] M. J. Kannwischer et al., “Improving software quality in cryptography standardization projects,” in IEEE EuroS&P Workshops, 2022.

[4] NIST, “Post-Quantum Cryptography: FIPS 203, 204, 205,” National Institute of Standards and Technology, 2024.

[5] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” Nature, vol. 549, pp. 188-194, 2017.

[6] M. Webber et al., “The impact of hardware specifications on reaching quantum advantage in the fault tolerant regime,” AVS Quantum Science, 2022.

[7] ANSSI, “Avis relatif a la migration vers la cryptographie post-quantique,” Agence nationale de la securite des systemes d’information, 2024.

[8] NIST, “Transition to Post-Quantum Cryptography Standards,” NIST IR 8547, 2024.

[9] BSI, “Kryptographische Verfahren: Empfehlungen und Schlussellangen,” BSI TR-02102-1, 2024.

[10] M. Mosca, “Cybersecurity in an era with quantum computers: will we be ready?” IEEE Security & Privacy, vol. 16, no. 5, pp. 38-41, 2018.